

List Processing and Pattern Matching

Learning Outcomes

by the **End** of the **Lecture**, **Students** that **Complete** the **Recommended Exercises** should be **Able** to:

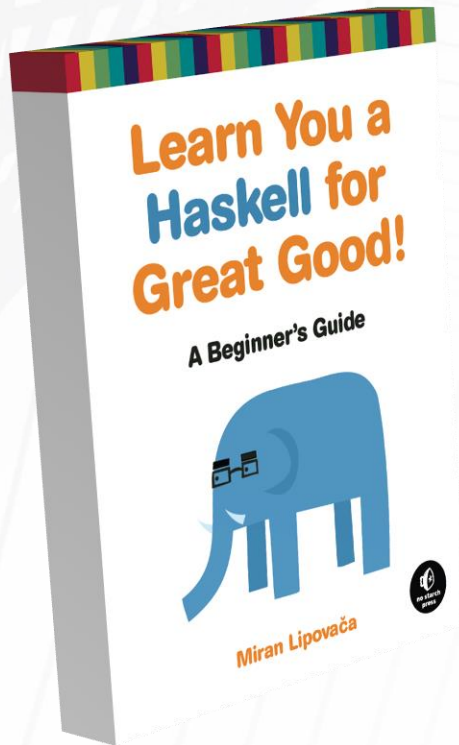
Use Lists of Tuples for Record Access and Manipulation

Contrast Ephemeral and Persistent Data Structures

Practice List Comprehensions as Database Queries

Explain the Process of Pattern Matching

Reading and Suggested Exercises



Read the following Chapter(s):

**Chapter 4:
"Pattern matching"**

Lists of Tuples as Databases

as noted previously, the **Tuple** types in **Haskell**
can serve the same **Purpose** as **Structs** in **C**
(i.e., **Tuples** can be used to **Define Custom Types**)

Database Relations (Tables) are **Sets of Tuples (Rows)**
but we can use **Lists of Tuples** for a **Similar Purpose**

Lists of Tuples as Databases

consider, as a clarifying **Example**, a **System** for **Video Rentals** (which used to be an actual thing)

Queries (i.e., **Requests for Information**) and **Updates** (i.e., **Modifications**) would correspond to **Functions** that **Require a Relation** as at least one of the **Arguments**

Lists of Tuples as Databases

```
type Client = Int
type Video = [Char]
type Rental = (Client, Video)
type Database = [Rental]
```

```
database :: Database
```

```
database = [ (763547, "The Thing"), (929845, "Escape from New  
York"), (929845, "The Thing"), (181014, "Big Trouble in Little  
China") ]
```

Lists of Tuples as Databases

What would be the **Type Declarations** for the
Following Queries and Updates?

Which Videos are on **Loan** to a specific **Client**?

Which Clients have **Borrowed** a specific **Video**?

How Many Videos are on **Loan** to a specific **Client**?

Has a specific **Video** been **Loaned** to any **Client**?

Loan a specific **Video** to a specific **Client**

Return a specific **Video** from a specific **Client**

Lists of Tuples as Databases

Which Videos are on **Loan** to a specific **Client**?

```
queryVideos :: [Rental] -> Client -> [Video]
```

Which Clients have **Borrowed** a specific **Video**?

```
queryClients :: [Rental] -> Video -> [Clients]
```

Lists of Tuples as Databases

How Many Videos are on **Loan** to a specific **Client**?

```
queryNumLoan :: [Rental] -> Client -> Int
```

Has a specific **Video** been **Loaned** to any **Client**?

```
queryIsLoan :: [Rental] -> Video -> Bool
```

Lists of Tuples as Databases

Loan a specific **Video** to a specific **Client**.

```
updateLoan :: [Rental] -> Client -> Video -> [Rental]
```

Return a specific **Video** from a specific **Client**.

```
updateReturn :: [Rental] -> Client -> Video -> [Rental]
```

Lists of Tuples as Databases

Considering the Referential Transparency
associated with **Haskell**, can you **Anticipate** any
Difficulty with the **Updates** (i.e., the latter two)?

Lists of Tuples as Databases

in **Imperative** and **Procedural Languages**,
Data Structures are **Ephemeral**

Ephemeral Data Structures are **Destroyed** by **Updates**
(or, if you prefer, you may **Consider** the **Structures**
subjected to an update to have **Mutated**)

Lists of Tuples as Databases

**in Functional Programming Languages,
Data Structures are Persistent**

Persistent Data Structures are Immutable

consequently, **Operations Do Not Modify** the
Existing Data Structure and, instead, **Operations**
produce **New Structures** (i.e., **Transformations**)

n.b., garbage collection can be used to ensure that memory is not exhausted

Lists of Tuples as Databases

now **Implement** using **List Comprehensions**
(and the **Generate-Test-Transform** pattern)

Which Videos are on **Loan** to a specific **Client**?

Which Clients have **Borrowed** a specific **Video**?

How Many Videos are on **Loan** to a specific **Client**?

Has a specific **Video** been **Loaned** to any **Client**?

Loan a specific **Video** to a specific **Client**

Return a specific **Video** from a specific **Client**

Lists of Tuples as Databases

Which Videos are on Loan to a specific Client?

```
queryVideos :: [Rental] -> Client -> [Video]
```

```
    queryVideos db who  
= [vid | (cli, vid) <- db, cli == who]
```

Which Clients have Borrowed a specific Video?

```
queryClients :: [Rental] -> Video -> [Clients]
```

```
    queryVideos db what  
= [cli | (cli, vid) <- db, vid == what]
```

Lists of Tuples as Databases

How Many Videos are on Loan to a specific Client?

```
queryNumLoan :: [Rental] -> Client -> Int  
  
    queryNumLoan db who  
= length [vid | (cli, vid) <- db, cli == who]
```

Has a specific Video been Loaned to any Client?

```
queryIsLoan :: [Rental] -> Video -> Bool  
  
    queryIsLoan db what  
= [cli | (cli, vid) <- db, vid == what] /= []
```

Lists of Tuples as Databases

Loan a specific **Video** to a specific **Client**.

```
updateLoan :: [Rental] -> Client -> Video -> [Rental]
```

```
updateLoan db cli vid  
= (cli, vid) : db
```

Return a specific **Video** from a specific **Client**.

```
updateReturn :: [Rental] -> Client -> Video -> [Rental]
```

```
updateReturn db cli vid  
= [x | x <- db, x /= (cli, vid)]
```

Polymorphism

if the **Tuple Arity** is known, our **Queries**
(which are **Essentially Selectors**) can be **Generic**

```
queryVideos :: Database -> Client -> [Video]
```

```
    queryVideos db cli'  
= [vid | (cli, vid) <- db, cli == cli']
```

...could also be **Accomplished** with the **Generic...**

```
findSecond :: [(k, v)] -> k -> [v]
```

```
    findSecond list key  
= [y | (x, y) <- list, x == key]
```

Polymorphism

```
findSecond :: [(k, v)] -> k -> [v]
```

```
    findSecond list key  
= [y | (x, y) <- list, x == key]
```

What Parameter Types would Match this Definition?

```
[(Client, Video)] -> Client -> [Video]
```

```
[(ID, Student)] -> ID -> [Student]
```

k and **v** would be Bound to these Types

Polymorphism

Most of the Built-In Prelude Functions are of a Polymorphic Function Type that can be Assessed using the `:t` Operation

```
:t length  
length :: [a] -> Int
```

```
:t head  
head :: [a] -> a
```

```
:t tail  
tail :: [a] -> [a]
```

Languages with Pattern Matching can Recognize Specific Arguments or Formats, allowing functions to be created from Separate Definitions for several Specific, Individual Cases

there was, for instance, another **Alternative** for **Implementing** the `implies` **Function** (from the branching lecture), using **Pattern Matching**

Pattern Matching

first **Create the Type Declaration:**

```
implies :: Bool -> Bool -> Bool
```

then **Create the Function Definition:**

```
implies True True = True  
implies True False = False  
implies False True = True  
implies False False = True
```

n.b., when this function is called on some arguments, each pattern is checked (from top to bottom) to see if the arguments match the definition

Pattern Matching

Pattern Matching can also be used to
Deconstruct Arguments into **Components**

Any of the following **Patterns** can be **Matched**

Literal
Variable
Wildcard
Tuple Pattern
Applied Constructor

`True, 'A', 9`
`w, x, y, z`
`—`
`(p1, p2, ...)`
`(x : _)`

n.b., the applied constructor will be discussed in more detail shortly, but for now you can consider an expression like `(h : t)` to be a function call `buildListFrom(h, t)`

Does the Argument Match the Pattern?

Argument

[1, 2, 3]

Pattern

a:(b:c)

**If the Answer is "Yes" then What Values
are Bound to Each Component?**

n.b., you can confirm your answer in GHCi by checking the expression

let pattern = argument for errors

Pattern Matching

Argument `[1, 2, 3]` **Matches Pattern** `a:(b:c)`

$$\begin{aligned} & [1, 2, 3] \\ &= (1 : [2, 3]) \\ &= (1 : (2 : [3])) \end{aligned}$$

What Values are Bound to Each Component?

$$a = 1 \quad b = 2 \quad c = [3]$$

Does the Argument Match the Pattern?

Argument

`[1, 2]`

Pattern

`a:(b:c)`

**If the Answer is "Yes" then What Values
are Bound to Each Component?**

n.b., you can confirm your answer in GHCi by checking the expression

`let pattern = argument` for errors

Does the Argument Match the Pattern?

Argument
[1]

Pattern
a:(b:c)

**If the Answer is "Yes" then What Values
are Bound to Each Component?**

n.b., you can confirm your answer in GHCi by checking the expression

let pattern = argument for errors

Does the Argument Match the Pattern?

Argument
`(1, 2, [3, 4])`

Pattern
`(a, b, c)`

**If the Answer is "Yes" then What Values
are Bound to Each Component?**

n.b., you can confirm your answer in GHCi by checking the expression

`let pattern = argument` for errors